

Tree of Thoughts: Deliberate Problem Solving with LLMs

Jane Tenecota-Villa, Jade Lee, Lauren Ah-Hot · Based on Yao et al., NeurIPS 2023 · github.com/yunseo2/deep-learning-tree-of-thought

1. Introduction

Large language models (LLMs) generate text autoregressively, committing to tokens left-to-right with no ability to backtrack. While effective on straightforward tasks, this mechanism struggles on problems requiring planning, exploring alternatives, or recovering from early mistakes.

Yao et al. introduced **Tree of Thoughts (ToT)** in their NeurIPS 2023 paper, a framework that treats LLM inference as a search over a tree of intermediate reasoning steps. Rather than committing to a single reasoning chain as in Chain-of-Thought (CoT) prompting, ToT generates multiple candidate "thoughts," evaluates each as sure/maybe/impossible, and uses BFS or DFS to navigate the tree with backtracking. The paper reports 74% success on Game of 24 vs. 7.3% (IO) and 4% (CoT) with GPT-4. Our project re-implements BFS-based ToT on Game of 24 using GPT-4o and extends the work with a few-shot vs. zero-shot evaluator ablation.

2. Chosen Result

We targeted the Game of 24 main result (Table 2): the IO/CoT/ToT success rate comparison, which directly supports the paper's central claim that search-based reasoning substantially outperforms standard prompting. The paper reports IO: 7.3%, CoT: 4%, ToT (b=5): 74%. We also aimed to reproduce the error analysis from Figure 3(b), which shows ~60% of CoT failures occur at the very first reasoning step, evidence that strict left-to-right decoding struggles on tasks requiring exploration. Beyond replication, we designed an evaluator ablation (few-shot vs. zero-shot) to isolate how much of ToT's performance comes from evaluator quality versus the search structure itself.

3. Methodology

Dataset and Model. We randomly sampled 50 puzzles from the 4nums.com Game of 24 dataset (hard games, indices 901-1,000), reducing from the paper's 100 due to budget constraints. We used GPT-4o via the OpenAI API. The paper used a May 2023 GPT-4 checkpoint; GPT-4o is the current successor and is both more capable and more cost-efficient.

ToT Implementation. We implemented BFS with depth $T=3$ (three intermediate arithmetic equations) and beam size $b=5$. At each tree node a *propose prompt* asks the model to enumerate candidate next steps given the remaining numbers. A separate *value prompt* classifies each candidate as sure/maybe/impossible with respect to reaching 24. We sample values 3 times per thought and combine scores to retain the top-b candidates per level.

Baselines. IO prompting (5-shot, direct answer) and CoT prompting (5-shot with 3 intermediate equations) serve as baselines, each sampled 5 times per puzzle. A solution is correct if it is a valid arithmetic equation using each input number exactly once and evaluating to 24, verified automatically by evaluating the generated expression through a program.

Modifications from the paper. (1) 50 vs. 100 puzzles due to API cost. (2) GPT-4o instead of GPT-4. (3) We wrote our own 7-example few-shot evaluator rather than reusing the paper's prompts. (4) Evaluator ablation: over 20 puzzles, the proposer and beam were held fixed while comparing our few-shot evaluator against a zero-shot evaluator, measuring both accuracy and wall-clock time per problem.

4. Results & Analysis

Method	Success Rate	Cost / Problem	Time / Problem
IO (5-shot)	10.0%	~\$0.0024	~1.2s
CoT (5-shot)	24.0%	~\$0.0022	~1.3s
ToT BFS (b=5)	88.0%	~\$0.1020	~86.0s

Table 1: Re-implementation results on 50 Game of 24 puzzles (GPT-4o).

Main results. Our re-implementation confirms the paper's key finding: ToT (88%) >> CoT (24%) >> IO (10%). Absolute numbers are higher than the paper's (74% / 4% / 7.3%), which we attribute to GPT-4o being a stronger model than the original May 2023 GPT-4 checkpoint. However, the relative ordering and large performance gap between ToT and standard prompting methods are fully replicated.

Evaluator ablation. This ablation was motivated by an implementation mistake: our first version used a zero-shot evaluator because the paper does not emphasize how important the few-shot examples are. After reviewing the official code, we realized the evaluator uses few-shot prompting and ran a direct comparison on 20 puzzles. Swapping our 7-example few-shot evaluator for a zero-shot one dropped accuracy from 95% to 25%. The zero-shot evaluator over-pruned "maybe" states, collapsing promising search paths too early. Runtime fell slightly (97s to 70s) but at a severe accuracy cost. The few-shot evaluator adds ~\$0.08 per problem yet gains 70 percentage points, a clear cost-effective win.

Cost analysis. ToT costs roughly 45x more per problem than IO or CoT (~\$0.10 vs. ~\$0.002) due to its search-based structure. However, our implementation is substantially cheaper than the original paper's setup, which reported roughly \$0.74 per Game of 24 problem using the May 2023 GPT-4 API. Using GPT-4o, our full 50-puzzle run cost about \$5 total. Cost per correct solution is more comparable across methods, since IO and CoT spend many attempts on failed solutions.

5. Reflections

Replication holds. Tree of Thoughts (ToT) consistently outperforms both CoT and IO on Game of 24, preserving the same qualitative ordering reported by Shunyu Yao et al. On our 50-problem sample, we observe 88% (ToT) vs. 24% (CoT) vs. 10% (IO). One notable difference is that the original paper reports CoT (4%) performing below IO (7.3%), which the authors attribute to CoT committing early to incorrect reasoning paths, while random direct guessing in IO occasionally succeeds by chance. We do not reproduce this inversion, likely because GPT-4o follows intermediate reasoning prompts more reliably than the original May 2023 GPT-4 checkpoint.

The few-shot evaluator matters. Removing few-shot demonstrations from the evaluator prompt caused performance to drop sharply from 95% to 25%. While the original paper does not explicitly isolate this component, our results suggest that evaluator quality is a major contributor to ToT's success, rather than just a small implementation detail. In particular, the evaluator's ability to correctly classify ambiguous "maybe" states depends heavily on concrete examples.

Cost is the main tradeoff. The largest downside of ToT is its search overhead. Exploring and evaluating multiple reasoning paths dramatically improves accuracy, but also increases runtime and API usage compared to standard prompting methods. Still, newer models like GPT-4o already make this tradeoff much more practical than it was at the time of the original paper, and future models will likely continue reducing both inference time and cost. This suggests that search-based reasoning

methods like ToT may become increasingly practical as model efficiency improves.

Future work. On the search side, replacing BFS with DFS and early stopping on high-confidence paths could preserve much of the accuracy while reducing cost, since BFS expands all candidates at every level. On the prompting side, it remains unclear whether 7 few-shot evaluator examples are actually necessary; testing different example counts (e.g., 3, 5, 7) could identify a cheaper configuration that still preserves most of the accuracy gain. Both directions point toward the broader challenge of making ToT more practical while maintaining its reasoning performance.

6. References

- [1] Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., & Narasimhan, K. (2023). Tree of Thoughts: Deliberate Problem Solving with LLMs. *NeurIPS 2023*. arXiv:2305.10601.
- [2] OpenAI (2024). GPT-4o. <https://openai.com/index/hello-gpt-4o/>
- [3] 4nums.com Game of 24 dataset / ToT code. <https://github.com/princeton-nlp/tree-of-thought-llm>